

Part III: Horizontal Scaling with Auto Scaling

Objective

The objective of Part III was to eliminate the bottleneck discovered in Part II by deploying the same Go-based product search service using horizontal scaling with ECS Fargate, an Application Load Balancer, and Auto Scaling.

The service logic remained unchanged:

- 100,000 in-memory products
- Linear search checking 100 products per request
- Same load test that previously broke the system

The goal was to demonstrate that horizontal scaling allows the system to handle the load that previously caused performance degradation.

Architecture Overview

The deployed infrastructure included:

1. Application Load Balancer (ALB)

- Distributes incoming HTTP requests
- Routes traffic to healthy ECS tasks
- Performs health checks on `/health`

2. Target Group

- Target type: IP (required for Fargate)
- Protocol: HTTP
- Port: 8080
- Health check path: `/health`
- Health check interval: 30 seconds
- Healthy threshold: 2 consecutive successes

3. ECS Service with Auto Scaling

- Metric: Average CPU Utilization
 - Initial target: 70%
 - Min tasks: 2
 - Max tasks: 4
 - Cooldown: 300 seconds
-

CS6650LB

▼ Details

Load balancer type

Application

Scheme

Internet-facing

Status

Active

Hosted zone

Z35SXDOTRQ7X7K

VPC

vpc-0066616eef14843a

Availability Zones

subnet-0896257e31b1b937d

us-east-1b (use1-az6)

subnet-02a7de3e45c5393c4

us-east-1e (use1-az3)

subnet-0311aa54acc957e7d

us-east-1f (use1-az5)

subnet-03ae1a26015aff4b0

us-east-1d (use1-az2)

subnet-038daa4562d946ec3

us-east-1a (use1-az4)

subnet-0c528894a7f60df10

us-east-1c (use1-az1)

Load balancer IP address type

IPv4

Date created

February 22, 2026, 23:59 (UTC-05:00)

Load balancer ARN

arn:aws:elasticloadbalancing:us-east-1:430206776298:loadbalancer/app/CS6650LB/53f38e9dd1183bee

DNS name info

CS6650LB-1742596039.us-east-1.elb.amazonaws.com (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Listeners and rules (1)

Info

Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

Filter listeners

Protocol:Port

Default action

Rules

ARN

Security policy

Default SSL/TLS certificate

mTLS

Trust store

HTTP:80

• Forward to target group

CS6650TG: 1 (100%)

Target group stickiness: Off

1 rule

ARN

Not applicable

Not applicable

Not applicable

Not applicable

CS6650TG

Actions

Details

am:aws:elasticloadbalancing:us-east-1:430206776298:targetgroup/CS6650TG/f6620212e72a536cd

Target type

IP

Protocol : Port

HTTP: 8080

Load balancer

CS6650LB

Protocol version

HTTP1

VPC

vpc-0066616eef14843a

2

Total targets

2

Healthy

0

Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

► Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

IP address

Port

Zone

Health status

Health status details

Administrative ov...

Override details

Anomaly detectio...

172.31.33.124

8080

us-east-1b (use1-az6)

Healthy

-

No override

No override is current...

Normal

172.31.61.150

8080

us-east-1e (use1-az3)

Healthy

-

No override

No override is current...

Normal

2 / 14

Resource map Info

View, explore, and troubleshoot your load balancer's architecture.

Overview

Unhealthy target map

Show resource details

CS6650LB

Last fetched seconds ago

Export

Listeners (1)

HTTP:80

1 rule

Rules (1)

Priority default
Forward to target group

Conditions (If)
If no other rule applies

Target groups (1) Info

IP, HTTP
CS6650TG

2 targets

2

0

0

0

0

0

Targets (2)

172.31.33.124

Port 8080

Healthy

172.31.61.150

Port 8080

Healthy

HTTP

http://CS6650LB-1742596039.us-east-1.elb.amazonaws.com/health

Save

Share

GET

http://CS6650LB-1742596039.us-east-1.elb.amazonaws.com/health

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description	...	Bu
	Key	Value	Description		

Body

Cookies

Headers (4)

Test Results

200 OK • 148 ms • 142 B •

Raw

Preview

Visualize

1 OK

Baseline: 20 Users (70% CPU Target)

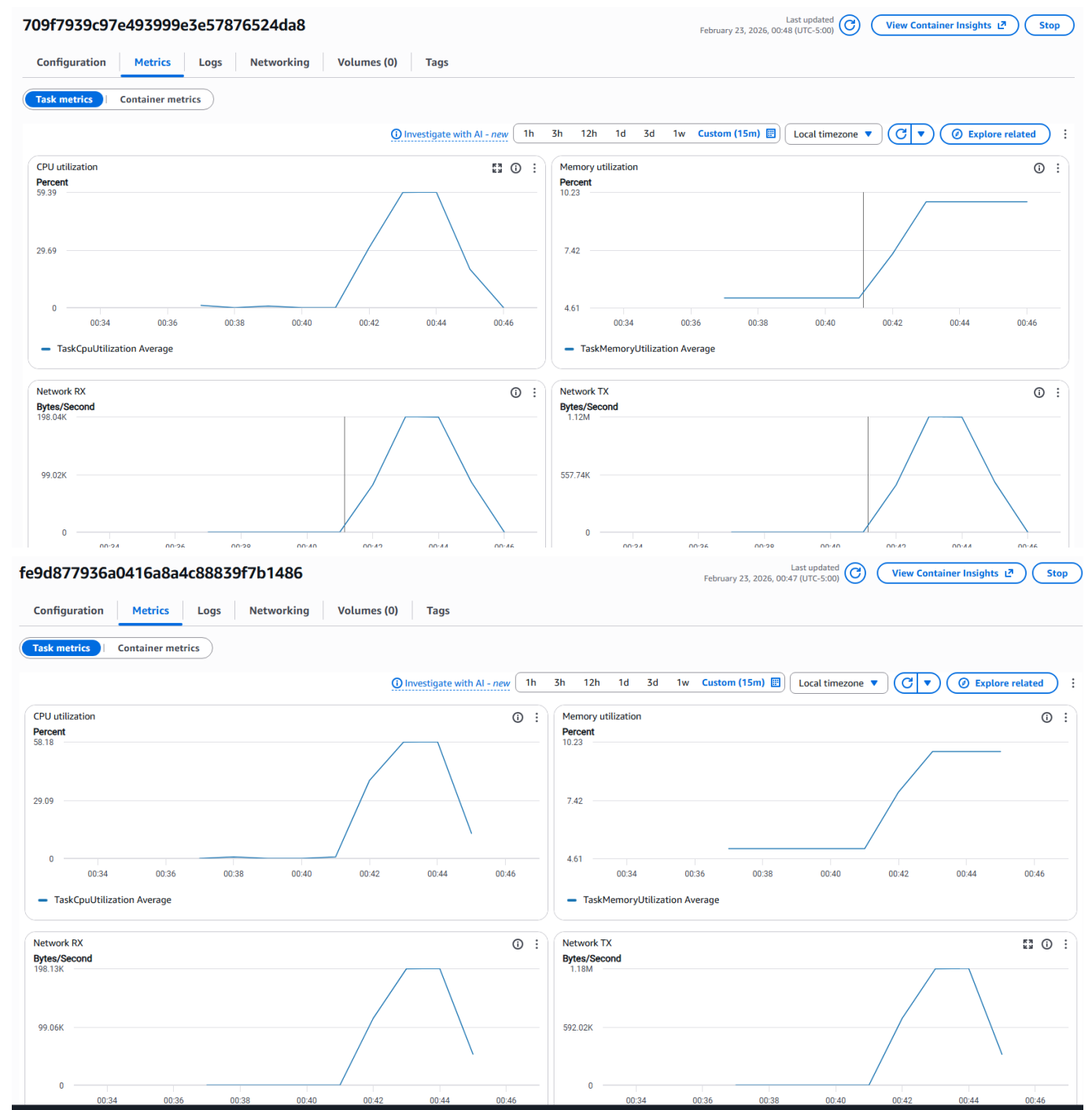
Test Configuration

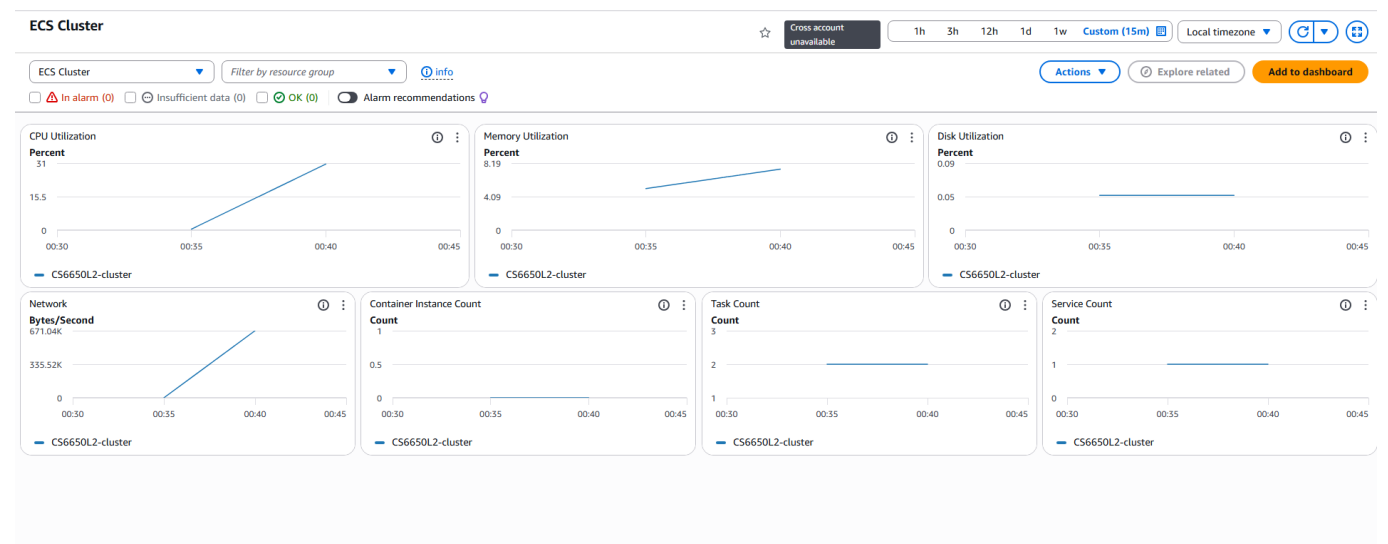
- Users: 20
- Duration: 3 minutes
- CPU target: 70%

Observations

CPU Utilization

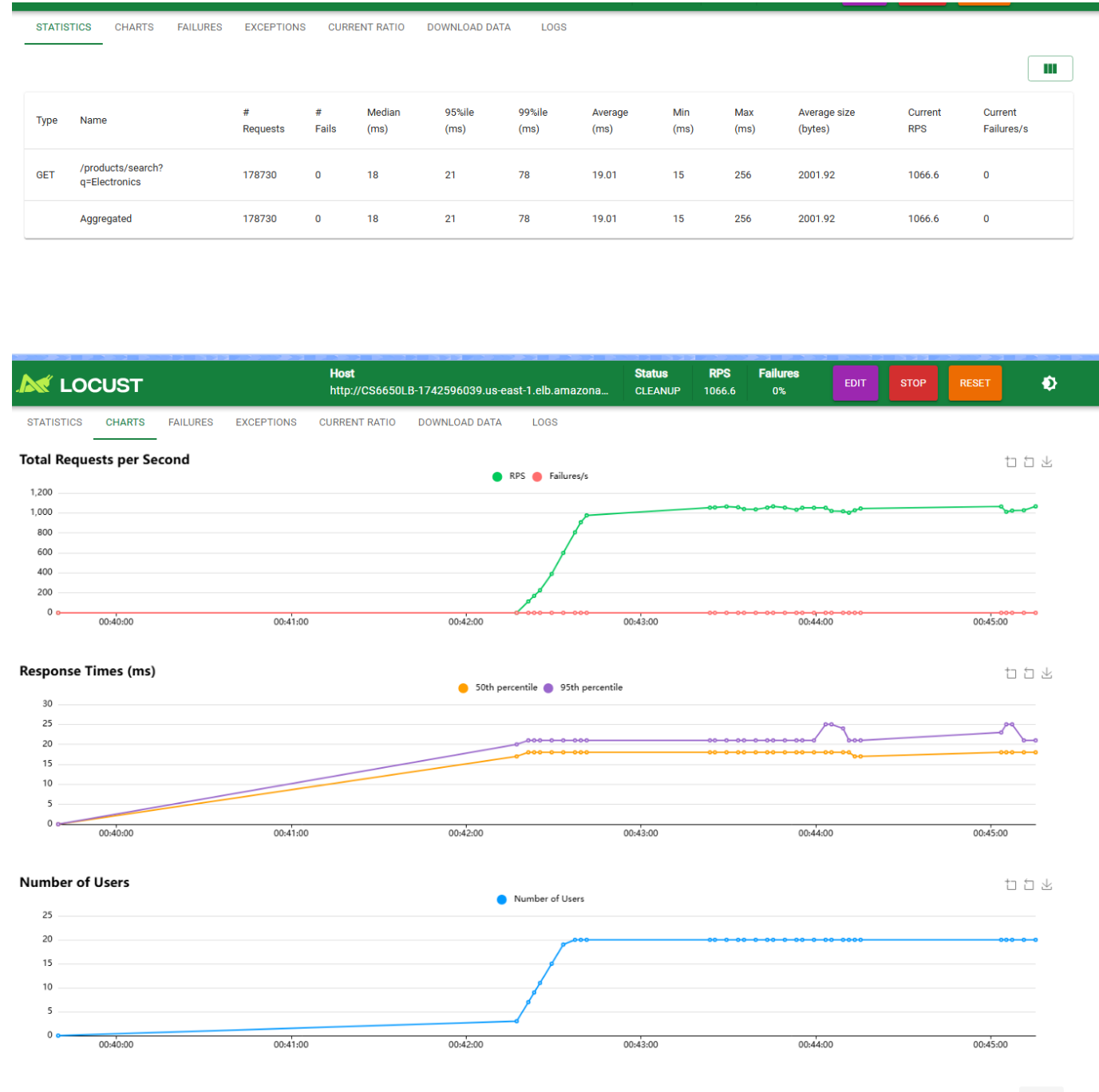
- Peak CPU per task: ~58–60%
- Did not cross 70%





Load Test Results

- RPS $\approx$ 1066
- Average latency $\approx$ 19 ms
- Failures: 0



Load Test Results

- Peak RPS $\approx$ 2177
- Total requests: 336,650
- Median latency: 19 ms
- 95th percentile: 41 ms
- 99th percentile: 110 ms
- Failures: 0

 LOCUST

Host

http://CS6650LB-1742596039.us-east-1.elb.amazona...

Status

STOPPED

RPS

2177.7

Failures

0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

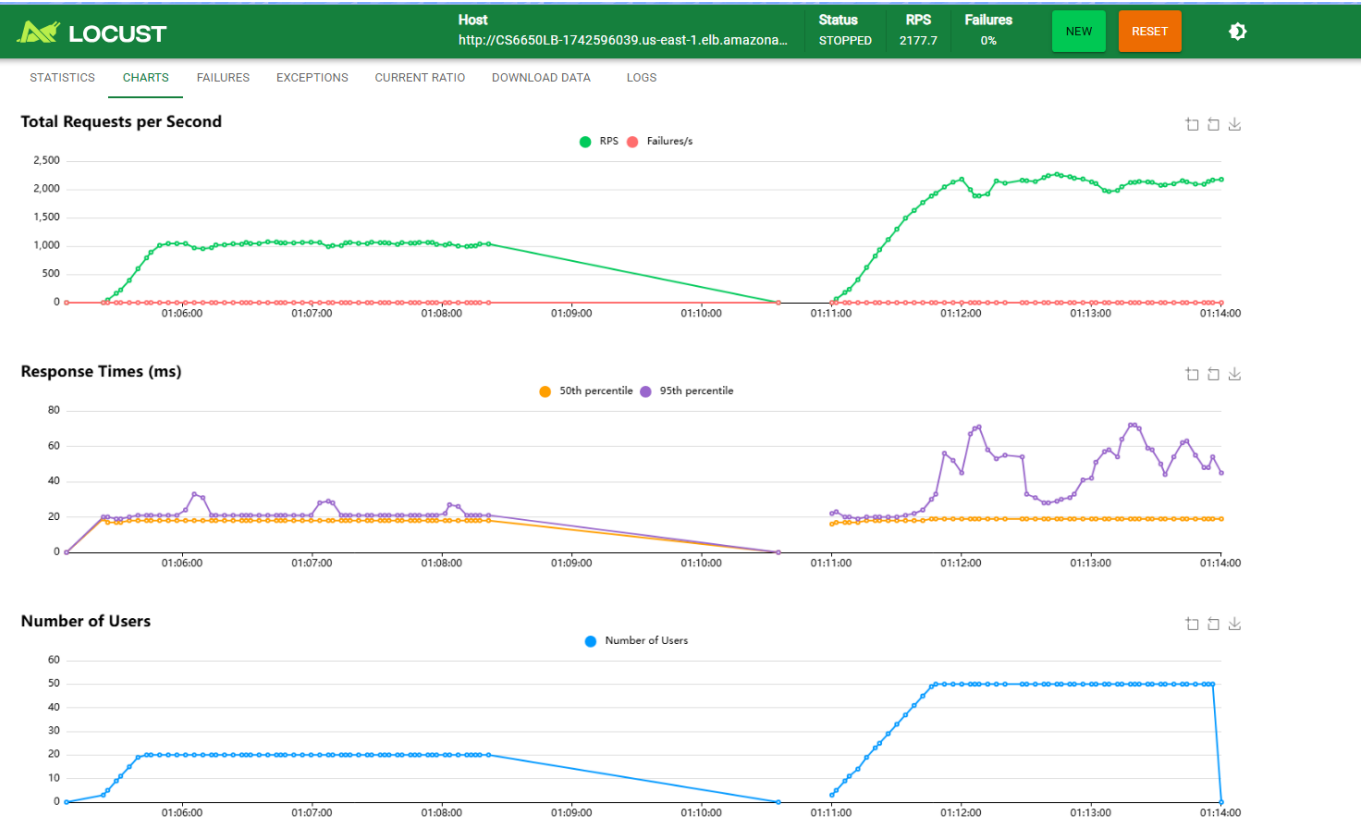
EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

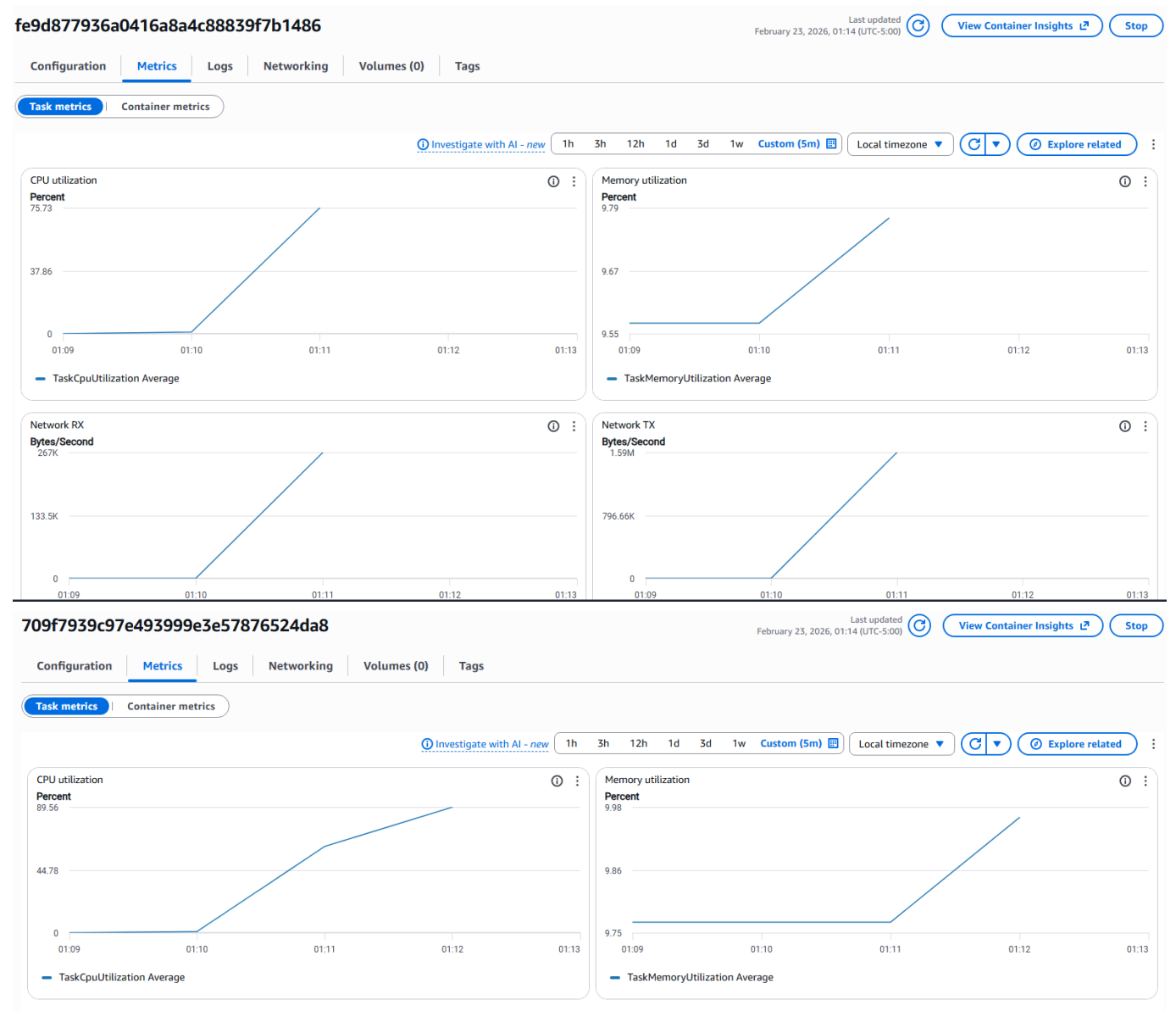
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search?q=Electronics	336650	0	19	41	110	22.76	15	259	2001.9	2177.7	0
	Aggregated	336650	0	19	41	110	22.76	15	259	2001.9	2177.7	0



CPU Triggered Scaling

CPU per task increased significantly:

- Task CPU peaked around 75%
- Another task peaked around 89%
- This exceeded the 45% threshold.



Auto Scaling Behavior

ECS Service Overview showed:

- Desired tasks: 4
- Running tasks: 4
- Previously: 2 tasks
- Newly created: 2 additional tasks

Service overview Info

Status

Active

Tasks (4 Desired)

0 Pending | 4 Running

Task definition: revision

CS6650L2-task:4

Deployment status

Success

Health and metrics

Tasks

Logs

Deployments

Events

Configuration and networking

Service auto scaling

Event history

Tags

Tasks (1/4)

February 23, 2026, 01:17 (UTC-5:00)

Last updated

Stop

Filter tasks by property or value

Filter desired status

Any desired status

Filter launch type

Any launch type

Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started at	Container instan...	Launch type
58e3b6b66d5c4fe184230...	Running	Running	CS6650L2-task:4	Unknown	1 minute ago	ecs-svc/60383704266...	45 seconds ago	-	Fargate
709f7939c97e493399e3e...	Running	Running	CS6650L2-task:4	Unknown	1 hour ago	ecs-svc/60383704266...	1 hour ago	-	Fargate
eef797e311364a2eb4c8f...	Running	Running	CS6650L2-task:4	Unknown	1 minute ago	ecs-svc/60383704266...	46 seconds ago	-	Fargate
fe9d877936a0416a8a4c8...	Running	Running	CS6650L2-task:4	Unknown	1 hour ago	ecs-svc/60383704266...	1 hour ago	-	Fargate

Containers for task 58e3b6b66d5c4fe184230d6268f29d98

Target Group Update

After scaling:

- Total targets: 4
- Healthy: 4
- Unhealthy: 0

This confirms 2 new tasks were launched, 2 new targets were registered, and health checks passed successfully.

CS6650TG

Actions

Details

arn:aws:elasticloadbalancing:us-east-1:430206776298:targetgroup/CS6650TG/6620212e72a536cd

Target type

IP

Protocol : Port

HTTP: 8080

Protocol version

HTTP1

VPC

vpc-0066616eef1484a3a

IP address type

IPv4

Load balancer

CS6650LB

4

Total targets

4

Healthy

0

Unhealthy

0

Unused

0

Initial

0

Draining

0

Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (4)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

< 1 >

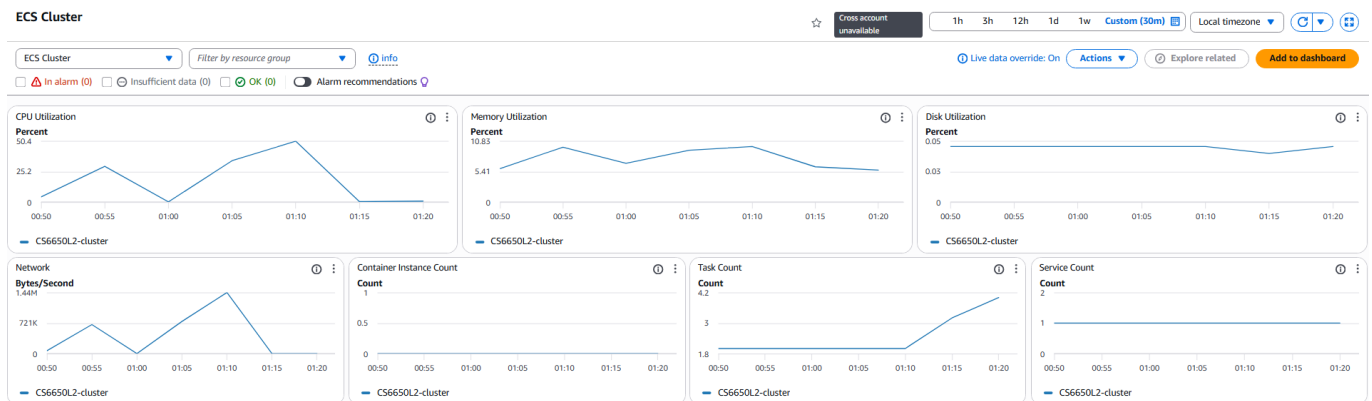
IP address	Port	Zone	Health status	Health status details	Administrative override	Override details	Anomaly detectio...
172.31.10.230	8080	us-east-1c (...)	Healthy	-	No override	No override is currently active on target	Normal
172.31.65.233	8080	us-east-1f (...)	Healthy	-	No override	No override is currently active on target	Normal
172.31.33.124	8080	us-east-1b (...)	Healthy	-	No override	No override is currently active on target	Normal
172.31.61.150	8080	us-east-1e (...)	Healthy	-	No override	No override is currently active on target	Normal

System Behavior During Scaling

Sequence of events:

1. User load increased to 50
2. CPU crossed 45%
3. Auto Scaling policy triggered

4. ECS launched 2 new tasks
5. Target Group registered new IP addresses
6. ALB redistributed traffic
7. Latency stabilized
8. No failures occurred



This demonstrates reactive horizontal elasticity.

Resilience Testing

To validate fault tolerance, one ECS task was manually stopped during active load.

Failure Injection

While the 50-user load was running:

- One task was manually stopped
- Status transitioned to Deactivating → Stopped

Target Group Reaction

Target group showed:

- 1 target in "Draining"
- 4 healthy
- 0 unhealthy

This proves the load balancer gracefully deregistered the stopped task and active connections were drained properly.

Automatic Replacement

ECS detected desired count mismatch and:

- Launched a new task
- Registered new IP in target group
- Restored total healthy targets

Health and metricsTasksLogsDeploymentsEventsConfiguration and networkingService auto scalingEvent historyTags

Tasks (1/5)

Filter tasks by property or value

Filter desired statusAny desired status

Filter launch typeAny launch type

Last updatedFebruary 23, 2026, 01:30 (UTC-5:00)

Stop

Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started at	Container instan...	Launch type
58e3b6b6d5c4fe184230...	Running	Running	CS6650L2-task:4	Unknown	13 minutes ago	ecs-svc/60383704266...	13 minutes ago	-	Fargate
ba52d453f750400b8550...	Running	Running	CS6650L2-task:4	Unknown	1 minute ago	ecs-svc/60383704266...	39 seconds ago	-	Fargate
eef797e311364a2eb4c8f...	Running	Running	CS6650L2-task:4	Unknown	13 minutes ago	ecs-svc/60383704266...	13 minutes ago	-	Fargate
fe9d877936a0416a8a4c8...	Running	Running	CS6650L2-task:4	Unknown	1 hour ago	ecs-svc/60383704266...	1 hour ago	-	Fargate
709f7939c97e493999e3e...	Deactivating	Stopped	CS6650L2-task:4	Unknown	1 hour ago	ecs-svc/60383704266...	1 hour ago	-	Fargate

Details

arn:aws:elasticloadbalancing:us-east-1:430206776298:targetgroup/CS6650TG/6620212e72a536cd

Target typeIP

Protocol : PortHTTP: 8080

Protocol versionHTTP1

VPCvpc-0066616eef1484a3a

IP address typeIPv4

Load balancerCS6650LB

5Total targets

4Healthy0Anomalous

0Unhealthy

0Unused

0Initial

1Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

TargetsMonitoringHealth checksAttributesTags

Registered targets (5) Info

Anomaly mitigation: Not applicableDeregisterRegister targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

< 1 >

IP address	Port	Zone	Health status	Health status details	Administrative override	Override details	Anomaly detection...
☐ 172.31.85.69	8080	us-east-1d (...)	Healthy	-	No override	No override is currently active on target	Normal
☐ 172.31.10.230	8080	us-east-1c (...)	Healthy	-	No override	No override is currently active on target	Normal
☐ 172.31.65.233	8080	us-east-1f (...)	Healthy	-	No override	No override is currently active on target	Normal
☐ 172.31.33.124	8080	us-east-1b (...)	Healthy	-	No override	No override is currently active on target	Normal
☒ 172.31.61.150	8080	us-east-1e (...)	Draining	Target deregistration i...	No override	No override is currently active on target	Normal

Scaling activities

Find scaling activity

< 1 ... >

Resource ID	Scalable dimension	Status	Status message	Start time	End time	Description	Not scaled reason	Cause
service/CS6650L2-cluster/CS6650L2	ecs:service:DesiredCount	Successful	Successfully set desired count to 3. Change successfully fulfilled by ecs.	Feb 23, 2026, 01:46	Feb 23, 2026, 01:46	Setting desired count to 3.	-	monitor alarm TargetTracking-serAlarmLow-d6b74821-8d37-45af-triggered policy Cs6650 scaling
service/CS6650L2-cluster/CS6650L2	ecs:service:DesiredCount	Successful	Successfully set desired count to 4. Change successfully fulfilled by ecs.	Feb 23, 2026, 01:16	Feb 23, 2026, 01:17	Setting desired count to 4.	-	monitor alarm TargetTracking-serAlarmHigh-43ad3143-9387-46ee-triggered policy Cs6650 scaling
service/CS6650L2-cluster/CS6650L2	ecs:service:DesiredCount	Successful	-	Feb 23, 2026, 00:57	-	Attempting to scale due to alarm triggered	AlreadyAtDesiredCapacity	monitor alarm TargetTracking-serAlarmLow-bd4e78f-a9fc-4447-ef-triggered policy Cs6650 scaling
service/CS6650L2-cluster/CS6650L2	ecs:service:DesiredCount	Failed	-	Feb 23, 2026, 00:23	-	Attempting to scale due to alarm triggered	AlreadyAtMinCapacity	monitor alarm TargetTracking-serAlarmLow-bd4e78f-a9fc-4447-ef-triggered policy Cs6650 scaling

Load Continuity

Locust metrics showed:

- RPS remained stable (~2177)
- Failures = 0
- No latency spike

This confirms zero downtime during instance failure.

How Horizontal Scaling Solved the Part II Bottleneck

In Part II:

- Single task handled all requests
- CPU saturated
- Response times increased dramatically
- System broke under load

In Part III:

- Load distributed across multiple tasks
- CPU per instance reduced
- Auto Scaling added capacity dynamically
- Latency remained stable
- Zero failures occurred

The bottleneck was compute capacity. Horizontal scaling solved it by distributing load.

Role of Each Component

Application Load Balancer

- Distributes traffic across healthy tasks
- Performs health checks
- Removes unhealthy instances automatically

Target Group

- Maintains registered backend instances
- Tracks health status
- Handles connection draining

Auto Scaling

- Monitors CPU utilization
- Adds tasks when threshold exceeded
- Removes tasks when load decreases
- Maintains desired count

Together, these components create an elastic and resilient system.

Horizontal vs Vertical Scaling Trade-offs

Horizontal Scaling

Advantages:

- Fault tolerant
- Scales dynamically
- Handles instance failure gracefully
- Cloud-native approach

Disadvantages:

- More infrastructure complexity
- Requires load balancing
- Slightly higher networking overhead

Vertical Scaling

Advantages:

- Simpler configuration
- No load balancer needed

Disadvantages:

- Single point of failure
- Limited scaling ceiling
- Requires downtime for resizing

Modern distributed systems prefer horizontal scaling because it enables elasticity and resilience.

Predicted Scaling Behavior

- **CPU Target = 70%:** Scaling triggers only under heavy sustained load
 - **CPU Target = 50%:** Scaling triggers earlier, more aggressive expansion
 - **CPU Target = 90%:** Risk of latency spikes before scaling
 - **Gradual load increases:** Smooth scaling behavior
 - **Sudden spikes:** Short latency increase until new tasks become healthy
-

Creative Stress Testing Performed

To demonstrate deep understanding, the following were performed:

- Baseline test (20 users)
- Forced scaling test (50 users, 45% threshold)
- Manual failure injection
- Observed connection draining
- Validated automatic replacement
- Compared latency across scenarios

This validated:

- Elastic scaling
 - High availability
 - Self-healing behavior
 - Proper ALB integration
-

Final Conclusion

The service that broke in Part II now:

- Handles more than 2× the original load
- Scales from 2 to 4 tasks automatically
- Replaces failed instances automatically
- Maintains zero downtime
- Keeps latency stable
- Demonstrates true cloud-native horizontal elasticity

This approach is foundational to modern distributed systems because it enables systems to scale dynamically, recover automatically, and maintain availability under stress.